



Binary Trees.

[COMMENT] Complete the Table, Continue the correctness of the minimal length.

Arrays, Linked Lists, Heaps, and Hash Tables are all ways to maintain collections. Each offers a different trade-off in query running times. For some of them, we imposed good worst-case bounds, while for others we can only compute expected costs (Hash Tables), detail are supplied in Table 5.1. The obvious question is whether one could guarantee both fast access and update running times in the worst-case.

In the following two lectures, we will see that using Balanced Binary Search Trees, one can provide both access and insertion/deletion in $O(\log n)$.

Data Structure	Access	Max/min	Insert/Delete	Max/min
Array	Data 2	Data 3	Data 3	$O(\log n)$
Linked-List	$O(n)$	$O(n)$	$O(1)$	$O(\log n)$
Heap	$O(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Hash Table	$O(n)$	$O()$	$O()$	$O(\log n)$

Table 5.1: Running times respecting to data structure and task

1 Binary Trees and How to Encode Them

We have already seen, in heaps, that organizing our data in a graph-like structure can offer a speed advantage. For future applications, and in particular for maintaining data in sorted order, we will have to encode our data using binary trees. In contrast to heaps, these trees may not be almost complete and also have to support pointer manipulations, specifically placing a binary tree as a left or right subtree of a given node. To enable this, we will have to treat the **right**, **left**, and **parent** as variables, in contrast to heaps where they are determined completely by the node index. We begin this section by stating definitions.

Definition 1.

1. Binary Tree: A tree in which any vertex has at most two children.

2. A descendant of vertex x is a vertex in the subtree whose root is x . A left descendant of vertex x is either a vertex in the subtree whose root is the left child of x , or x 's left child.
3. An ancestor of x is a vertex to which x belongs as a descendant.
4. A leaf is a vertex without children.
5. Height of vertex x is the length of the longest simple path (without cycles) between x and one of the leaves. The height of leaf is 0.
6. Height of the tree is the height of its root, which is usually denoted by h .

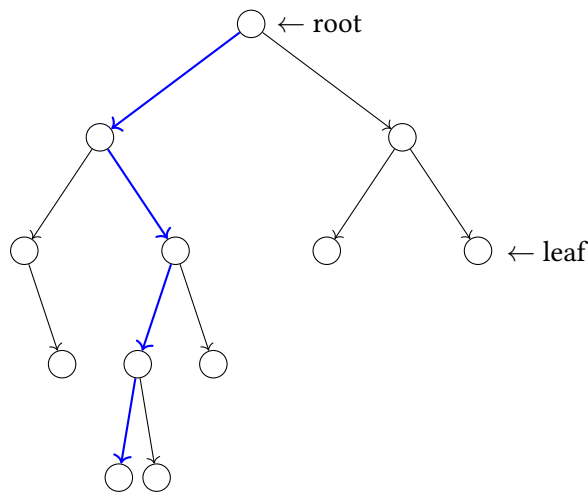


Figure 5.1: Example of a binary tree with 12 vertices. In blue, one of the longest paths from the root to one of the leaves, which is of length 4.

Question. What are the maximal and minimal heights of binary trees over n vertices?

Answer. The maximal height is $n-1$, and matches to trees in which any vertex has at most one child. The minimal height is $\log n$. Let's prove it by induction.

Denote by H_n the minimal height of a binary tree with n vertices. Observe that $H_{n+1} \geq H_n$. That's because adding a vertex to an n -size binary tree can't shorten any of the paths from root to leaves, particularly not the longest one.

1. Base. $n = 1$, The tree consists of a single vertex, which is both the root and a leaf. Thus, the longest path is of length 0, and also $\log 1 = 0$.
2. Assumption. Assume the correctness of the claim for any $m < n$.
3. Step. Consider a binary tree over n vertices with minimal height. The longest path consists of an edge between the root and one of its children. Denote

by x the size of the root's left subtree, so the size of root's right subtree is $n - x$. Thus we get:

$$H_n \geq 1 + \max(H_x, H_{n-x})$$

By the monotonic property of H_n , we have that the above is minimized when both $H_x = H_{n-x} = H_{n/2}$ if n is even, or $\max(x, n - x) = \lceil n/2 \rceil$ when n is odd. So we get:

$$\begin{aligned} &\geq 1 + H_{\lceil n/2 \rceil} \\ &= 1 + \log(\lceil \frac{n}{2} \rceil) = \log(n + 1) \end{aligned}$$

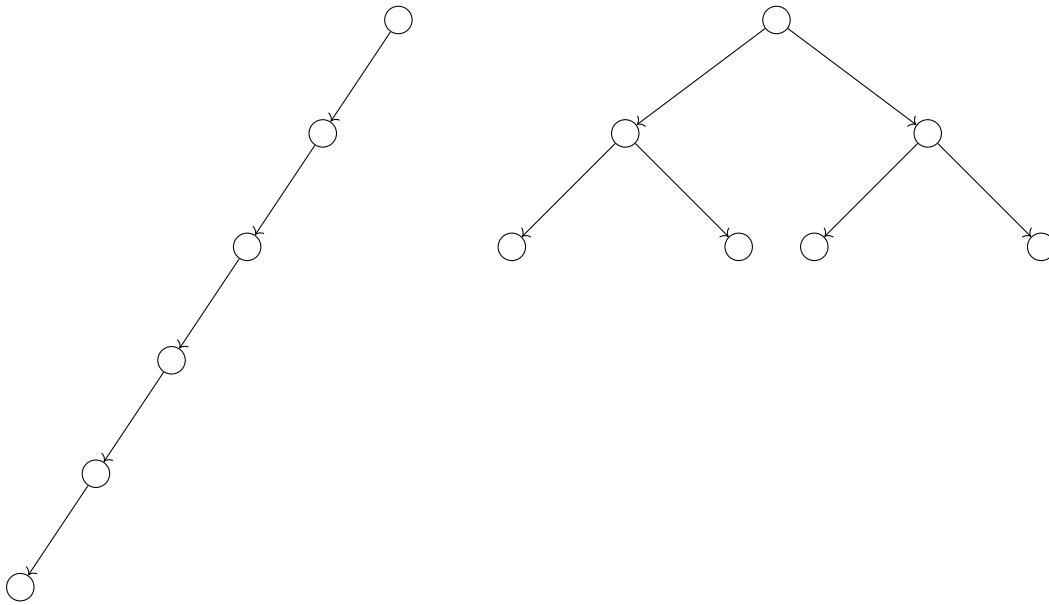


Figure 5.2: On the left, a binary tree with 6 vertices at height 5, while on the right, a binary tree with 7 vertices at height 2.

1.1 Encoding

We encode a binary tree by associating a field to each vertex x , representing its right, left children, and parent. We use the notation $x.\text{left}$ to refer to the left child of x , although the physical implementation may differ conceptually. For example, the way binary trees are implemented in Cormen is through 4 arrays. The first stores the value of x , while the others store pointers of specific types. For instance, the array `LEFT`, where `LEFT. $x$` stores the left child of x .

If nothing else has been mentioned, then we can assume that we can add additional fields to the vertices.

2 Binary Search Trees.

Now, we are ready to understand to store numbers (or keys), by binary trees.

Definition 2. Binary search tree (BST) is a binary tree which any node x of it:

1. Contains a field key, storing a number $x.\text{key}$.
2. Any left descendant y of x satisfies $y.\text{key} \leq x.\text{key}$.
3. Any right descendant y of x satisfies $y.\text{key} \geq x.\text{key}$.

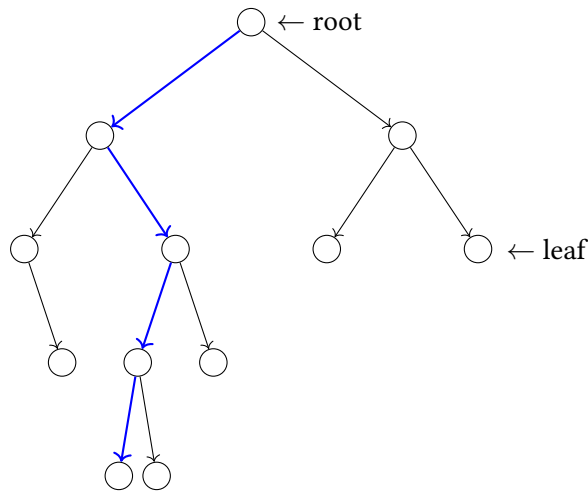


Figure 5.3: The heap after the exchange, at line (9). The induction assumption is applied over the right subtree, which is framed in gray.

Question. Let T be a binary search tree, Where are the minimum and maximum values of T ? (most left and right nodes).

Answer. Consider the algorithm which gets down by taking only left turns until hitting edge, as in Algorithm 1, Guideline for correctness: denote by z, r the minimal vertex by both value and height, or in other words, the highest vertex among the vertices with minimal value, and the root. Consider the path descending from r to z . Suppose that while walking through the path, we take a right turn at vertex y . Then, by the BST property, $y \leq z$. If $y < z$, then it contradicts z being minimal by value. Otherwise, if $y = z$, then it contradicts z being the highest vertex among the minimal vertices by value.

Question. Given that the value x is in T and that it's unique, how do you find the vertex that stores it?

Answer. Consider algorithm Line 2.

Definition 3. Let T be a binary search tree, and let x be a node belonging to it.

Data: T - tree

Result: pointer to vertex with minimum value in T

```
1 if  $T$  is empty then  
2   | return null  
3 end  
4 if  $T$ .left is empty then  
5   | return  $T$   
6 end  
7 return Min( $T$ .left)
```

Algorithm 1: Min

Data: T - tree, key - key to search for

Result: pointer to vertex with key equal to key

```
1 if  $T$  is empty then  
2   | return null  
3 end  
4 if  $T$ .key equals key then  
5   | return  $T$   
6 end  
7 if key is less than  $T$ .key then  
8   | return Search( $T$ .left, key)  
9 end  
10 else  
11   | return Search( $T$ .right, key)  
12 end
```

Algorithm 2: Search

1. The **predecessor** of x will be defined as a vertex y such that $y.key \leq x.key$ and $y.key$ is maximal among the nodes satisfying this condition. If we were to set the values of T in sorted order, then the predecessor of x would be located on its left.
2. The **successor** of x will be defined as y , where x is the predecessor of y .

Question.

Answer.

Data: x - vertex

Result: pointer to predecessor of x

```

1 if  $x.left$  is not empty then
2   | return  $\text{Max}(x.left)$ ;
3 end
4  $y \leftarrow x.parent$ ;
5 while  $y$  is not empty and  $x$  is  $y.left$  do
6   |  $x \leftarrow y$ ;
7   |  $y \leftarrow y.parent$ ;
8 end
9 return  $y$ ;

```

Algorithm 3: Predecessor

Last time, we saw some operations that can be performed on BSTs, and proved correctness for some of them. These were: $\text{Search}(x)$, $\text{Min}(T)$, $\text{Max}(T)$, $\text{Pred}(x)$, $\text{Succ}(x)$, $\text{Insert}(x)$. All of these operations take $O(h(T))$ in the worst case.

The main two operations that may cause problems are *Insert* and *Delete*, as they change the tree's height (consider inserting 81, 82, 83, 84 to our working example). To address this problem, we introduce another field: for each node v , add a field of $h(v)$ = the height of the subtree whose root is v . This allows us to maintain the AVL property:

AVL Tree. An AVL tree is a balanced BST, such that for any node x , its left and right subtrees' height differs in no more than 1. In other words:

$$|h(\text{left}(x)) - h(\text{right}(x))| \leq 1$$

This field allows us to calculate the Balance Factor for each node in $O(1)$:

Balance Factor. For each node $x \in T$, it's Balance Factor is defined

$$hd(x) := h(\text{left}(x)) - h(\text{right}(x))$$

In AVL trees, we would like to maintain $|hd(x)| \leq 1$

Example 1. For our working example, the node 60's hd is $h(50) - h(70) = 1$, and $hd(50) = h(45) - h(55) = -1$. You can check and see that this is an AVL tree.

So to make sure that we can actually maintain time complexity $O(\log(n))$, we would want to:

1. Show that for an AVL tree, $h(T) = \theta(\log(n))$ (If there's time left)
2. See how to correct violations in AVL property - using **rotations**
3. See how to *Delete* and *Insert*, while maintaining the height field.

2.1 Rotations

 Recitations/rotations.png

Rotations allow us to maintain the AVL property in $O(1)$ time (you have discussed this in the lecture - changing subtree's roots). In this schematic representation of rotations, x, y are nodes and α, β, γ are subtrees. Note that the BST property is maintained!

The Balance factor allows us to identify if the AVL property was violated, and moreover - the exact values of the bad Balance factors will tell us which rotations to do to fix this:

Taken from last year's recitation:

 Recitations/violations.png

Let us analyze one of these cases:

Example 2. Let us see why R rotation Fixes LL violation:

 Recitations/LLviolation.jpg

This is the general form of LL violations.

Let us analyze the heights of subtrees. Denote h the height of A_L before inserting v . So A_R 's height has also to be h . If its height were $h + 1$, the insertion would not have created a violation in B (A 's height would have stayed the same). If it were $h - 1$, the violation would have appeared first in A (not in B). Thus, A 's height is $h + 1$. B_R 's height is also h : If it was $h + 1$ or $h + 2$, no violation in B had occurred.

After the rotation, the tree looks like this: So all the nodes here maintain AVL property; why is it maintained?

Detect the violation in the following tree, and perform the necessary rotations:

Recitations/LLviolationfix.jpg

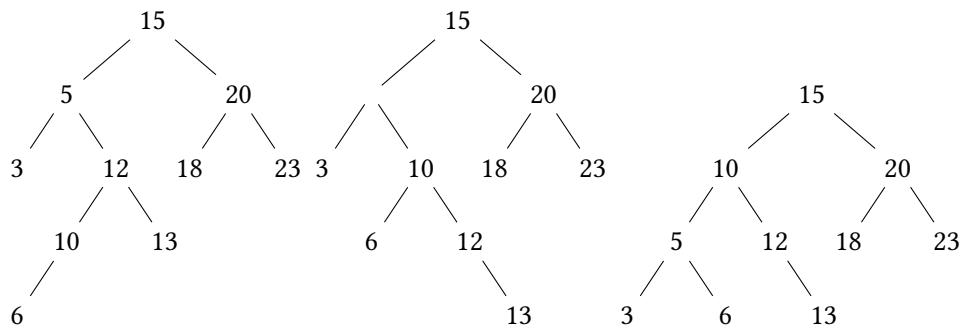


Figure 5.4: The first node in which a violation occurred is 5, this is an RL violation. Perform R rotation on the right child. Then perform L rotation on the root of the relevant subtree (5):

2.2 Delete, Insert.

The principles of the *Delete* and *Insert* operations are the same as in regular BST, but we will need to rebalance the tree in order to preserve AVL property. A single insertion or deletion may change the height difference of subtrees by at most 1, and might affect only the subtrees with roots along the path from r to the point of insertion/ deletion. More concretely - we will add a recursive operation of traversing the tree "back up" and checking violations. Had we found one - we will fix it using rotations. Since rotations can be done in $O(1)$, the entire correction process will take $O(\log(n))$, so we maintain a good time complexity.

```

1 Call Insert ( $r, x$ ) // (The standard insertion routine for BST)
2 Let  $p$  be the new node appended to the tree.
3 while  $p.parent \neq \emptyset$  do
4   | Check which of the four states we are in.
5   | Apply the right rotation.
6   | Update the height of each touched vertex
7   |  $p \leftarrow p.parent$ 
8 end
AVL Insert( $r, x$ )

```

2.3 Exam Question.

Consider the following question from 2018 exam.

(3) אתם מתבקשים לממש מבנה נתונים התומך בפעולות הבאות:

$\text{insert}(x)$: הכנסת המספר x למבנה, אם x לא נמצא בו כבר.

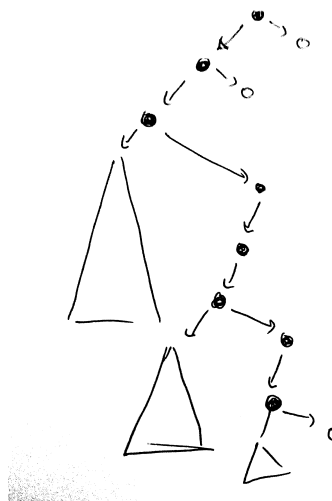
$\text{delete}(x)$: מחיקת x מהמבנה, אם x נמצא בו.

$\text{sum_over}(x)$: מחזיר את סכום כל המספרים במבנה שערכם גדול או שווה לזה של x .

סיבוכיות כל פעולה צריכה להיות $O(\log n)$, כאשר n הוא מספר האיברים הנוכחי במבנה.

Solution. How would it look like an algorithm which computes the 'sum over' query? Or for given x , which vertices have a key less than x ? Consider the path $v_1, v_2, v_3 \dots v_n$ on which the Search subroutine went over when looking for x . If $v_i.\text{key} < v_{i+1}.\text{key}$ then it means that x is greater than $v_i.\text{key}$. Furthermore, x is also greater than all the keys in the left subtree of v_i .

Let us transform that insight into an Algorithm. Define for each vertex a field that stores the summation of all its left subtree keys plus its own key; call it 'Leftsum'. Then computing the sumover query will be done by summing these fields of the vertices in the right turn on the searching path.



```

1  if  $r$  is None then
2    return 0
3  end
4  else
5    if  $x > r.\text{key}$  then
6      return  $r.\text{Leftsum} + \text{Sumover}(r.\text{right}, x)$ 
7    end
8    else
9      return  $\text{Sumover}(r.\text{left}, x)$ 
10   end
11 end

```

So it has left to show how one could maintain the tree and guarantee a logarithmic height. Consider a vertex v and suppose that both his direct children are correct AVL trees and have the correct value at the Leftsum field. We know that:

1. There is a rotation that balances the tree at the cost of $O(1)$ time.
2. All the descendants of v , at a distance at least two from it, will remain the same in the sense that their subtree has not changed.

Therefore we will have to recompute the Sumleft filed for only a $O(1)$ vertices (which are the children and the grandchildren of v previews the rotation). Each computation could be done in $O(1)$ time.

AVL-Leftsum Insert(r,x)	1	Call Insert (r, x) // (The standard insertion routine for BST)
	2	Let p be the new node appended to the tree.
	3	while $p.parent \neq \emptyset$ do
	4	Apply the right rotation.
	5	for any vertex v such that its pointers have changed, sorted by height do
	6	$v.Leftsum \leftarrow v.key + v.left.Leftsum + v.left.right.Leftism$
	7	end
	8	$p \leftarrow p.parent$
	9	end

Running time. The work is done over vertices along a single path, and on each vertex only $O(1)$ time is spent, Then we have that the total running time of the algorithm is proportional to the tree height. Combining the fact that we maintain the AVL property, it follows that the total time is $\Theta(\log n)$.

Appendix

2.4 AVL tree's height

Let n_h be the minimal number of nodes in an AVL tree of height h .
 n_h is strictly increasing in h .

Proof. Exercise. □

$$n_h = n_{h-1} + n_{h-2} + 1.$$

Proof. For an AVL tree of height 0, $n_0 = 1$, and of height 1, $n_1 = 2$ (by checking directly).

Let's look at a minimal AVL tree of height h . By the AVL property, one of its subtrees is of height $h - 1$ (WLOG - the left subtree) and by minimality, its left subtree has to have n_{h-1} nodes. T 's right subtree thus has to be of height $h - 2$: It can't be of height $h - 1$: $n_{h-1} > n_{h-2}$ by the previous theorem, and if the right subtree is of height $h - 1$ - we could switch it with an AVL tree of height $h - 2$, with fewer nodes - so less nodes in T , contradicting minimality. So the right subtree has n_{h-2} nodes (once again, by minimality), and thus the whole tree has $n_h = n_{h-1} + n_{h-2} + 1$ (added the root) nodes. □

$$n_h > 2n_{h-2} + 1 \quad h = O(\log(n))$$

Proof. Assume k is even (why can we assume that?). It can be shown by induction that:

$$n_h > 2n_{h-2} + 1 > 2(2n_{h-4} + 1) + 1 = 4n_{h-4} + (1+2) \dots > 2^{\frac{h}{2}} + \sum_{i=0}^{\frac{h}{2}-1} 2^i = \sum_{i=1}^{\frac{h}{2}} 2^i = \frac{2^{\frac{h}{2}+1} - 2}{2-1} = 2^{\frac{h}{2}+1} - 2$$

So $n_h \geq 2^{\frac{h}{2}} - 1$, thus

$$h \leq 2 \log(n_h + 1)$$

and for generall AVL tree with n nodes and height h :

$$h \leq 2 \log(n_h + 1) \leq 2 \log(n + 1) = O(\log(n))$$

□

Remark 1. In fact, one can show that $n_h > F_h$ and F_h is the h 'th Fibonacci number. Recall that $F_h = C(\varphi^h - (\psi)^h)$, and this gives a tighter bound on n_h .